SYSTEM PROGRAMMING

Week 6: Processes, fork, exec, wait & signals

VNU - UNIVERSITY *of* ENGINEERING *&* TECHNOLOGY

Agenda

01

Process Fundamentals

PID · PPID · states · process tree

02

Creating processes

fork() · return values · copy-on-write

03

Executing Programs

exec family · process replacement

04

Wait family & Zombies

waitpid() · status · zombie · orphan

05

Signals

SIGINT · SIGTERM · SIGCHLD · sigaction

06

Wrap-up

Lab tasks · recap · next week

01

Process Fundamentals

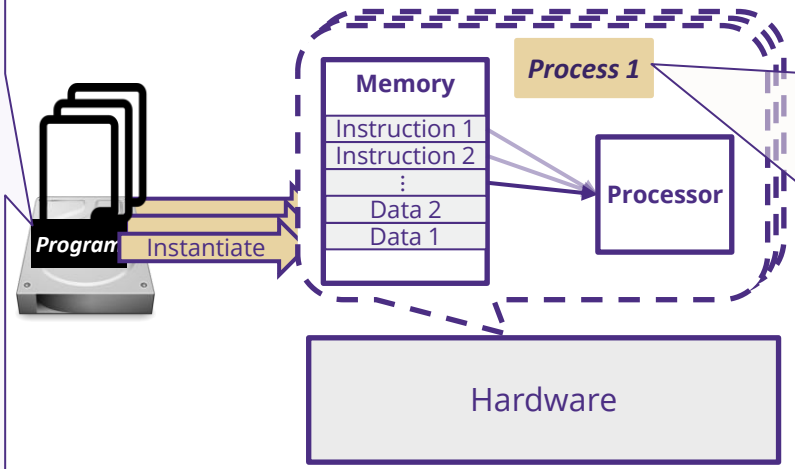
Program vs process · states · process tree

Program vs. Process

Program

A file stored on disk

- Executable file containing code and static data
- Passive - it just sits on disk
- One binary can launch many processes



Process

A running instance of a program

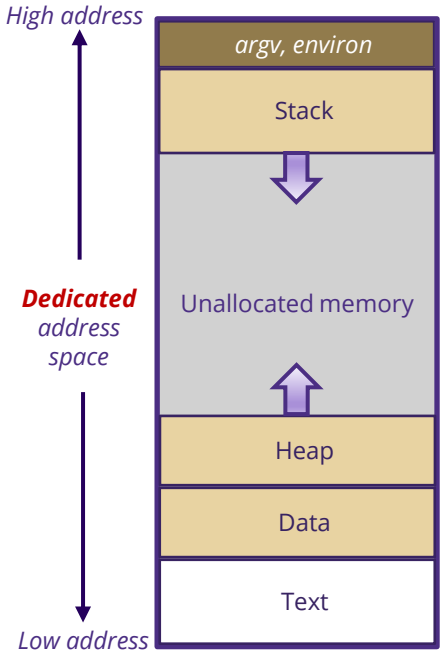
- Managed actively by kernel
- Has a unique PID and PPID
- Owns virtual address space
- Has its own CPU & kernel states (*process context*)



💡 If `ls` is open in two terminals → two processes, each with its own PID, memory space, and process context.

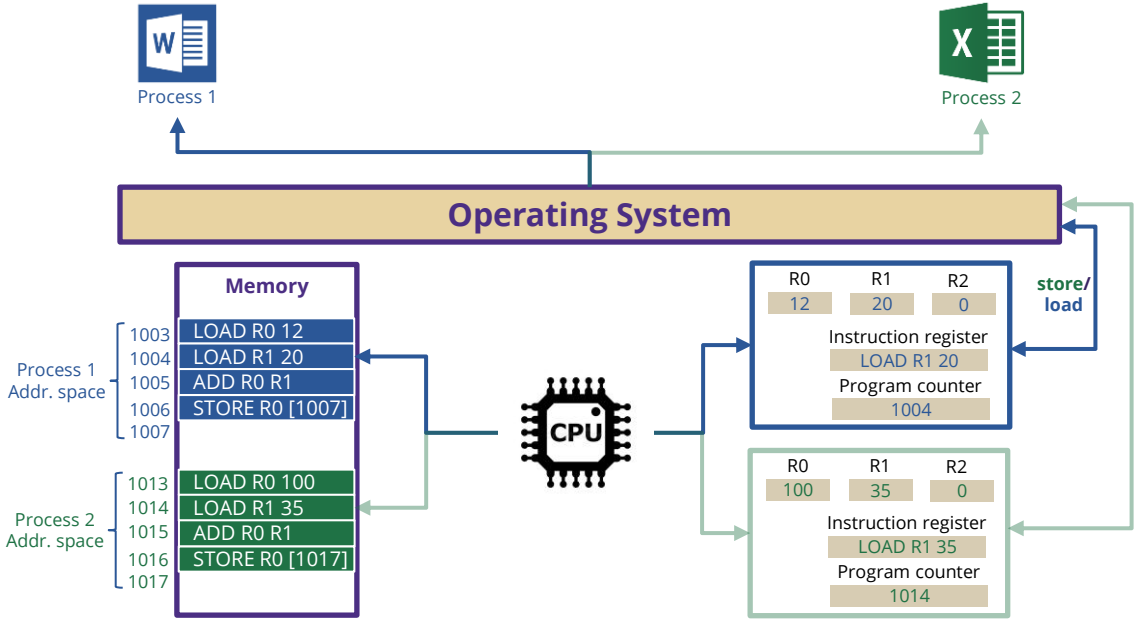
Process Memory

- Text: holds the program code
  - Read only to prevent self-modification
  - May be shared with other processes
- Data: holds global & static variables
  - Initialized
  - Uninitialized (technically initialized to 0)
- Stack: holds function's local variables
  - Grows & shrinks as the program executes
- Heap: holds dynamic data structures
  - `malloc()`, `calloc()`, `free()`, etc.



CPU States

> Linux runs multiple processes concurrently by sharing CPU time
  - OS must store the current CPU state before switching to another process.



Process Context

Definition *The complete set of information the kernel must save and restore in order to pause a process and resume it later - correctly.*

CPU Context (hardware state)

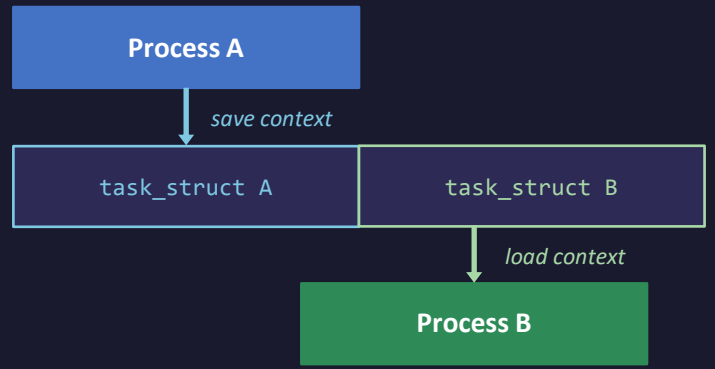
- Program Counter (PC) - address of next instruction
- Stack Pointer (SP) — top of current stack
- General-purpose registers — in-flight computation
- CPU flags — condition codes (zero, carry, overflow...)

OS Context (kernel-managed state)

- PID / PPID — identity and parent relationship
- Virtual memory map — page table pointer
- Open file descriptor table
- Working directory · credentials · signal dispositions

Context Switch

The kernel saves one process's context and loads another's. This creates the illusion that each process owns the CPU.



Linux stores each process context in a task_struct *(the Process Control Block).*

PID, PPID, and the process image

```
printf("PID=%ld PPID=%ld\n",
       (long)getpid(),
       (long)getppid());

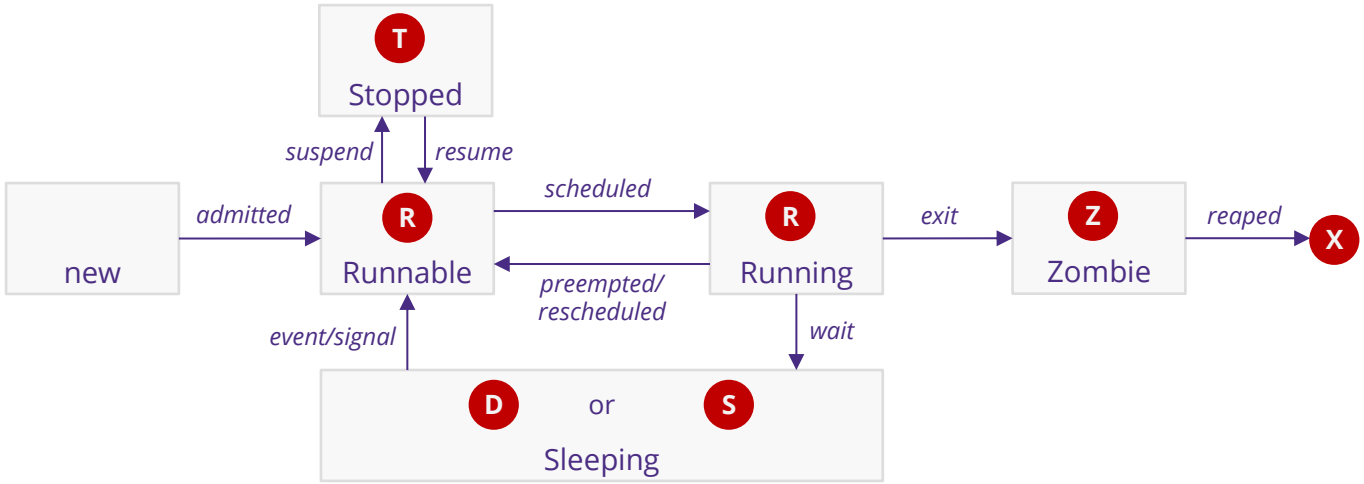
$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 2400  2381 Ss   bash
 2412  2400 S    ./worker
```

PID and PPID
A PID identifies a running process. PPID identifies its current parent.

Process image
The running program has code, data, stack, heap, and open descriptors.

Read identity, state, and command together when debugging a process tree.

Linux Process Life Cycle



R

Running/Runnable

On CPU or ready to run

S

Sleeping

Waiting for an event

D

Uninterruptible

Deep kernel / disk wait

T

Stopped

Paused by signal

Z

Zombie

Exited, not yet reaped

Process states in ps

```
ps -o pid,ppid,stat,cmd -p 2412

  PID  PPID STAT CMD
 2412  2400 S    ./worker

# Run repeatedly while the process works:
watch -n 0.5 'ps -o pid,ppid,stat,cmd -p 2412'
```

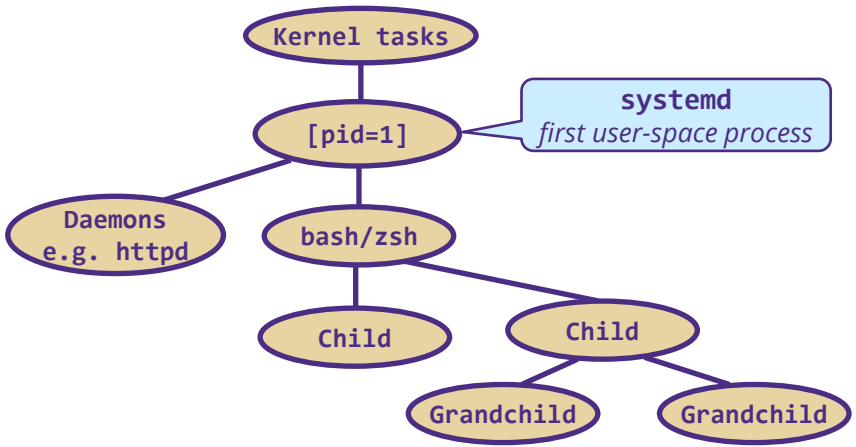
R: running or runnable
The process uses a CPU or waits to be scheduled.

S / T / Z
S sleeps while waiting for an event.
T is stopped.
Z has exited but awaits wait().

State letters describe a snapshot. A process can change state between two ps commands.

Linux Process Tree

Process Tree (`pstree -p`)



Hierarchy relationship

A parent process creates children processes → grandchildren processes → a tree of processes

Each user process has a PID and a parent PID (PPID)

Useful way of dividing a task

A server process is listening for client requests

- When a request arrives → creates a child process to handle it
- Parent continues listening for new requests

💡 Killing a parent process does not automatically kill its children

02

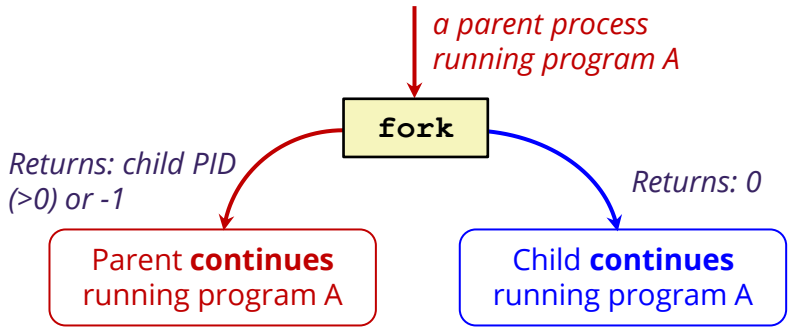
Creating processes

fork() · copy model · COW

fork() — One System Call, Two Processes

fork.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    // child: fork returned 0
    _exit(0);
} else {
    // parent: pid is child's PID
}
```



The Strange Truth

After one call to fork()

- The process is duplicated
- Parent & child continue from the **same instruction**
- Each sees a different return value

💡 After a failed exec() in the child, use _exit() to avoid flushing inherited stdio buffers.

The three outcomes of fork()

```
pid_t pid = fork();

if (pid == -1) {
    perror("fork" );
} else if (pid == 0) {
    /* child branch */
} else {
    /* parent branch */
    /* pid is the child PID */
}
```

Child: return value 0
The child continues at the instruction after fork().

Parent: child PID
The parent continues at the same place.
Scheduling decides which runs first.

A return of -1 means no child was created; handle the error in the parent.

What fork() Copies & Copy-on-Write

Logically Duplicated

- address space (stack, heap, globals)
- CPU state
- file descriptor table
- signal dispositions
- environment variables
- current working directory

Copy-on-Write (COW)

- Pages are not copied immediately
- Parent and child share physical pages
- A copy is made only when one side writes

`fork()` is much cheaper than it looks: spawning child processes is efficient even for large programs.

afterfork.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Before\n");
    fork();
    printf("After\n");
    return 0;
}
```

👉 How many times does “After” print?

Terminal: stdout is usually line-buffered. “Before” flushes at the newline; “After” appears twice.

👉 What changes if stdout is redirected to a file?

Redirected file: stdout is usually fully buffered. The unflushed “Before” is copied at fork(), so both processes may flush it. Try fflush(stdout) before fork().

Copy-on-write: separate values after a write

cow.c

```
int x = 5;
pid_t p = fork();
if (p == 0) {
    x = 9;
    printf("child: %d\n", x);
    _exit(0);
}
waitpid(p, NULL, 0);
printf("parent: %d\n", x);
```

Which value does each process print?

`child: 9`

`parent: 5`

The variables have the same name but belong to separate process address spaces.

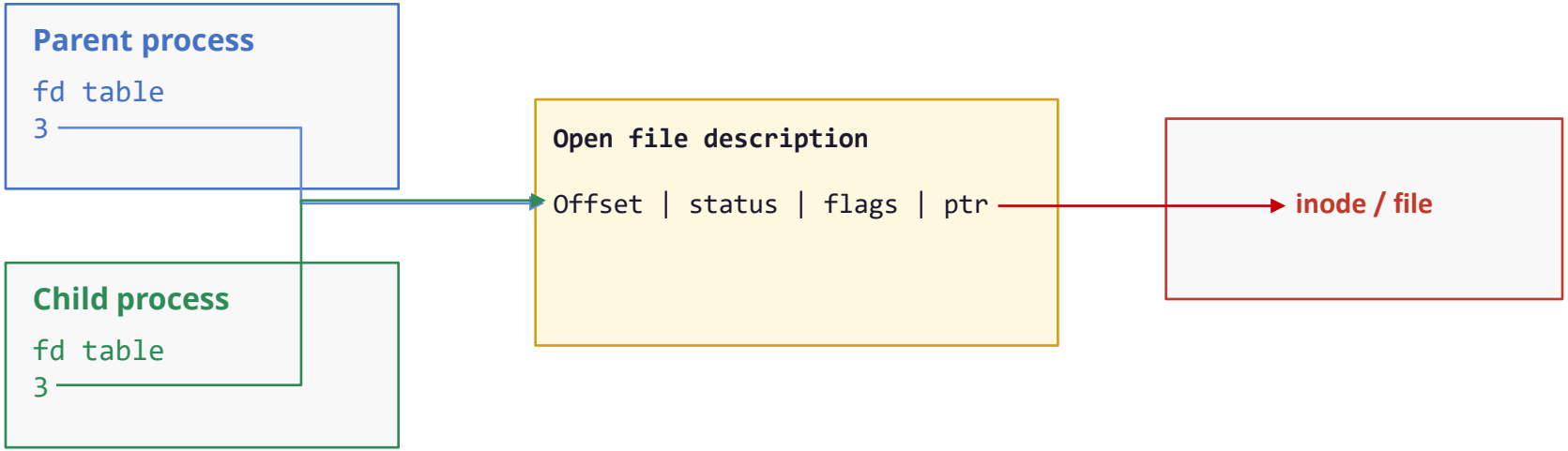
Memory model

Before write: same initial value

After write: independent memory

The kernel can share physical pages until one process changes a page.

File Descriptors After fork()



Key rule

Parent and child have separate `fd` tables, but entries pointing to the same file share the same kernel open file description - including the file offset.

⚠ This is one of the most important concepts in the course - shared file offsets after fork() produce surprising behavior.

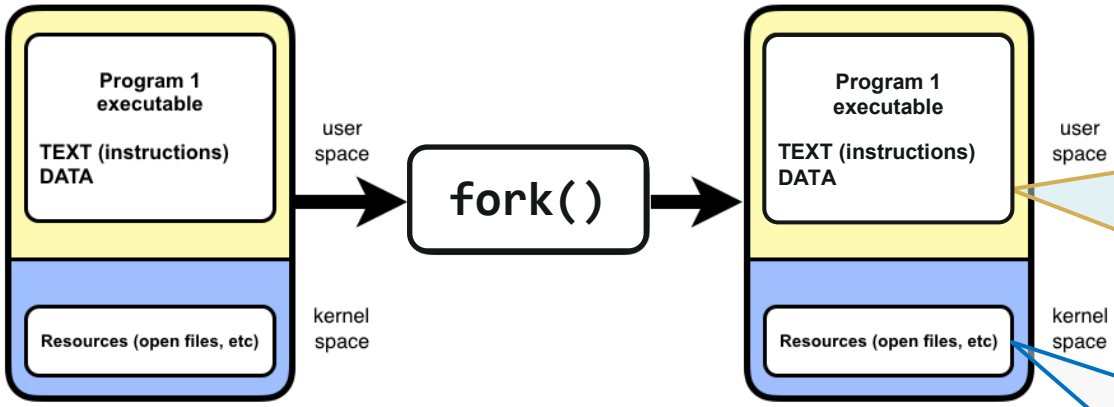
03

Executing Programs

exec family · tiny shell · builtins vs external commands

Replaces the Executable of the Current Process

exec() loads an executable file into an existing process, replacing the previous program.



What exec() replaces

- code
- data
- heap
- stack
- caught signal handlers reset to default

What exec() preserves

- PID
- PPID
- current working directory
- open fd's (unless `the close-on-exec flag is set`)
- ignored signals generally stay ignored

⚠ exec() NEVER returns on success

If `exec()` succeeds, the old program is gone → any code after `exec()` is failure handling.

fork() followed by execvp()

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork" );
    return 1;
}
if (pid == 0) {
    char *argv[] = {"ls", "-l", NULL};
    execvp(argv[0], argv);
    perror("execvp" );
    _exit(127);
}
waitpid(pid, NULL, 0);
```

Parent process
Keeps running the original program and waits for the child.

Child process
Starts as a copy after fork().
On success, execvp() replaces its program image.
On failure, execvp() returns.

The child PID stays the same across a successful execvp().

The exec family and process launch

```
execl(path,  arg0, arg1, ..., NULL);
execv(path,  argv);
execlp(file, arg0, arg1, ..., NULL);
execvp(file, argv);
```

- **l** = list of args (varargs)
- **v** = vector / argv[] array
- **p** = search PATH automatically



Typical launcher workflow

1. prepare an argv[] array
2. fork a child process
3. child calls execvp()
4. parent calls waitpid()
5. on execvp() failure: perror(), then _exit(127)

Built-ins vs. external commands

- Built-ins run inside the shell process: `exit`, `cd`
- External commands run in a child process: `ls`, `cat`

Why execvp() is the best choice for a shell

- Shell already parsed the command into `argv[]`
- User types `ls`, not full path → requires PATH searching

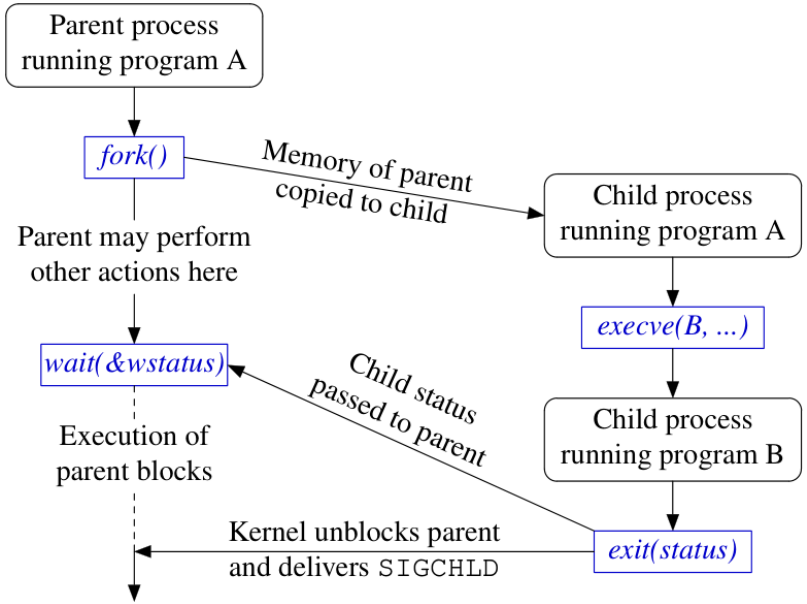
04

Wait family & Zombies

exit status · macros · zombie · orphan

The wait() family

wait() and waitpid() collect child exit status. With default options the parent blocks; WNOHANG checks without blocking.



Variations

```
wait(&status)
waitpid(pid, &status, 0)
waitpid(-1, &status, WNOHANG)  // non-blocking
```

Always use the macros

The status integer is encoded.
Do NOT decode it manually.

```
WIFEXITED / WEXITSTATUS
WIFSIGNALED / WTERMSIG
WIFSTOPPED / WSTOPSIG
```

Reading a child exit status

```
int status;
pid_t done = waitpid(pid, &status, 0);
if (done == -1) {
    perror("waitpid" );
} else if (WIFEXITED(status)) {
    printf("exit code: %d\n",
           WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("signal: %d\n",
           WTERMSIG(status));
}
```

waitpid(pid, ...)
Waits for one chosen child and returns its PID when the state is available.

Status macros
WIFEXITED: normal return or exit().
WEXITSTATUS: the exit code.
WIFSIGNALED: ended by a signal.

Read status through the macros. The integer itself is an encoded value.

Synchronization Example

child1.c

```
#include <stdio.h>
#include <unistd.h>
int main(void) {
  pid_t p = fork();
  if (p < 0) return 1;
  if (p == 0) puts("I am the child");
  else puts("I am the parent");
  return 0;
}
```

Could you guess the output?

```
I am the parent
I am the child
```

```
I am the child
I am the parent
```

How can the parent guarantee that the child prints first?

child2.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
  pid_t p = fork();
  if (p < 0) return 1;
  if (p == 0) {
    puts("I am the child");
    return 0;
  }
  waitpid(p, NULL, 0);
  puts("I am the parent");
}
```

Zombie & Orphan Processes

zombie.c

```
pid_t pid = fork();
if (pid == 0) { _exit(0); } // child exits

printf("child pid = %d\n", pid);
sleep(30);      // parent does not wait yet
// check: ps -o pid,ppid,stat,cmd -p <childpid>
```

Zombie (Z) process

- Child has already exited
- Parent has NOT called `waitpid()`
- Kernel keeps the exit record open
- The zombie uses a PID slot - not just memory
- **Fix**: always `waitpid()` your children

orphan.c

```
printf("Parent PID=%ld exiting\n", (long)getpid());
pid_t pid = fork();
if (pid == 0) {
    sleep(2);
    printf("Child PPID=%ld\n", (long)getppid());
    _exit(0);
}
exit(0);
```

Orphan process

- Parent exits before the child
- Linux reparents it to the nearest child subreaper, or to PID 1.
- Orphans continue running normally
- The new parent can reap it when it exits

💡 A zombie is not executing – it's just a record waiting for the parent to collect the exit status.

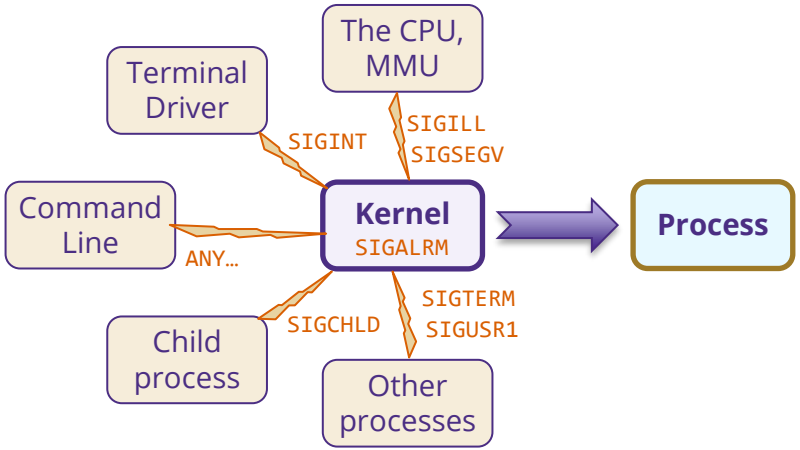
05

Signals

sigaction · SIGINT · safe handlers · shell behavior

Signals — Asynchronous Notifications

Signals are small messages that notify a process that an event of some type has occurred in the system.



ID	Name	Default Action	Corresponding Event
2	SIGINT	Terminate	Interrupt (e.g., Ctrl-c from keyboard)
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate & Dump	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

💡 The only information in a signal is its ID and the fact that it arrived.

Asynchronous

A signal can be delivered at any time, independent of the process's current execution flow

Sending and Receiving Signals

Sending signals

From within a program:`int kill(pid_t pid, int sig);`

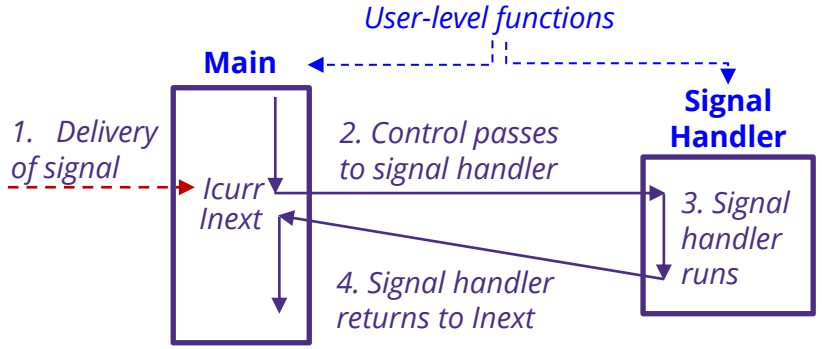
- Returns 0 if success, -1 if failure
- `int pid` - target process ID (must has matching UID, unless sender has elevated privileges)
- `int sig` - signal number or symbolic constant

Send yourself a signal: `int raise(int sig);`

Flow control on receipt of a signal

The kernel interrupts the target process and

- invokes the registered handler (if any), or
- performs the default action,
- after the handler returns, execution resumes where it was interrupted.



Legacy signal() interface

`handler_t signal(int sig, handler_t handler);`

- Returns a pointer to the previous signal handler
- `sig` - signal number or symbolic constant
- `handler` - pointer to signal handler function, or 1 of:
  - `SIG_IGN` - ignore signals
  - `SIG_DFL` - revert to the default action

Signal Handler Example

> What happens when you run the following code?

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int n = 0;

void sigint_handler(int sig) {
    printf("Handler catches signal %d\n", sig);
    n = 0;
}

void main() {
    signal(SIGINT, sigint_handler);
    for (int i = 0; i < 10; i++) {
        printf("Main count = %d\n", n++);
        sleep(1);
    }
}
```

KEY TAKEAWAYS

1. *fork() creates a child; both processes resume after the call with different return values.*
2. *Linux uses copy-on-write - memory pages are shared until one side writes.*
3. *File descriptors are copied but open file descriptions (offsets) may be shared.*
4. *exec() replaces the program image; it never returns on success. Use _exit() in child failure paths.*
5. *waitpid() collects child status. Always use the WIFEXITED/WIFSIGNALED macros.*
6. *Zombies happen when the parent forgets to waitpid().*
7. *A SIGTERM handler can set a flag; main code then performs safe cleanup.*

Next week: Pipes & Core IPC